

End-to-End Exploratory Data Analysis (EDA) on the Titanic Dataset

Project Objective: To perform a comprehensive, step-by-step exploratory data analysis to understand the key factors that influenced survival on the Titanic. This notebook will serve as a complete guide, covering data loading, cleaning, analysis, feature engineering, and visualization, with theoretical explanations at each stage.

Theoretical Concept: What is Exploratory Data Analysis (EDA)?

Exploratory Data Analysis is the crucial process of performing initial investigations on data to discover patterns, spot anomalies, test hypotheses, and check assumptions with the help of summary statistics and graphical representations. It is not about formal modeling or hypothesis testing; rather, it is about getting to know your data before you start building models.

Why is it important?

1. **Understand the Data:** It helps you understand the variables and their relationships.
2. **Data Cleaning:** It reveals missing values, outliers, and other inconsistencies that need to be handled.
3. **Feature Selection:** It helps identify which variables are the most important for your problem (feature engineering and selection).
4. **Assumption Checking:** It allows you to check assumptions that are required for certain machine learning models (e.g., normality, linearity).

Libraries Used: Pandas and Seaborn

- **Pandas:** This is a powerful Python library for data manipulation and analysis. It provides data structures like DataFrames, which are essential for working with tabular data. We used Pandas to load the dataset, handle missing values, and perform various data transformations.
- **Seaborn:** Built on top of Matplotlib, Seaborn is a statistical data visualization library. It provides a high-level interface for drawing attractive and informative statistical graphics. We used Seaborn to create various plots like countplots, histograms, boxplots, and barplots to visualize the distributions and relationships within the data.

Step 1: Setup - Importing Libraries

We'll start by importing the essential Python libraries for data manipulation (`pandas`, `numpy`) and visualization (`matplotlib`, `seaborn`).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set plot style for better aesthetics
sns.set(style='whitegrid')
```

Step 2: Data Loading and Initial Inspection

We'll load the dataset and take our first look at its structure, content, and overall health.

```
!git clone 'https://github.com/HarshvardhanSingh-13/Datasets'

Cloning into 'Datasets'...
remote: Enumerating objects: 303, done.ote: Counting objects: 100%
(61/61), done.ote: Compressing objects: 100% (55/55), done.ote: Total
303 (delta 14), reused 0 (delta 0), pack-reused 242 (from 1)

# Load the dataset from a URL
titanic_df = pd.read_csv('/content/Datasets/Titanic_Dataset/Titanic-
Dataset.csv')

# Display the first 5 rows
print("First 5 rows of the dataset:")
titanic_df.head()

First 5 rows of the dataset:

{"summary":{"\n  \"name\": \"titanic_df\",\n  \"rows\": 891,\n  \"fields\": [\n    {\n      \"column\": \"PassengerId\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 257,\n        \"min\": 1,\n        \"max\": 891,\n        \"num_unique_values\": 891,\n        \"samples\": [\n          710,\n          440,\n          841\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Survived\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n        \"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          1,\n          0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Pclass\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 1,\n        \"max\": 3,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          3,\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 891,\n        \"samples\": [\n          \"Moubarek, Master. Halim Gonios (\\\"William George\\\")\",\n          \"Kvillner, Mr. Johan Henrik Johannesson\"

```

```

],\n      \"semantic_type\": \"\", \n      \"description\": \"\"\n}\n  },\n  {\n    \"column\": \"Sex\", \n    \"properties\": {\n      \"dtype\": \"category\", \n      \"num_unique_values\": 2, \n      \"samples\": [\n        \"female\", \n        \"male\"\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }, \n    {\n      \"column\": \"Age\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 14.526497332334044, \n        \"min\": 0.42, \n        \"max\": 80.0, \n        \"num_unique_values\": 88, \n        \"samples\": [\n          0.75, \n          22.0\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }, \n      {\n        \"column\": \"SibSp\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 1, \n          \"min\": 0, \n          \"max\": 8, \n          \"num_unique_values\": 7, \n          \"samples\": [\n            1, \n            0\n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\"\n        }, \n        {\n          \"column\": \"Parch\", \n          \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 0, \n            \"min\": 0, \n            \"max\": 6, \n            \"num_unique_values\": 7, \n            \"samples\": [\n              0, \n              1\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n          }, \n          {\n            \"column\": \"Ticket\", \n            \"properties\": {\n              \"dtype\": \"string\", \n              \"num_unique_values\": 681, \n              \"samples\": [\n                \"11774\", \n                \"248740\"\n              ], \n              \"semantic_type\": \"\", \n              \"description\": \"\"\n            }, \n            {\n              \"column\": \"Fare\", \n              \"properties\": {\n                \"dtype\": \"number\", \n                \"std\": 49.693428597180905, \n                \"min\": 0.0, \n                \"max\": 512.3292, \n                \"num_unique_values\": 248, \n                \"samples\": [\n                  11.2417, \n                  51.8625\n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\"\n              }, \n              {\n                \"column\": \"Cabin\", \n                \"properties\": {\n                  \"dtype\": \"category\", \n                  \"num_unique_values\": 147, \n                  \"samples\": [\n                    \"D45\", \n                    \"B49\"\n                  ], \n                  \"semantic_type\": \"\", \n                  \"description\": \"\"\n                }, \n                {\n                  \"column\": \"Embarked\", \n                  \"properties\": {\n                    \"dtype\": \"category\", \n                    \"num_unique_values\": 3, \n                    \"samples\": [\n                      \"S\", \n                      \"C\"\n                    ], \n                    \"semantic_type\": \"\", \n                    \"description\": \"\"\n                  }, \n                  {\n                    \"column\": \"Survived\", \n                    \"properties\": {\n                      \"dtype\": \"category\", \n                      \"num_unique_values\": 2, \n                      \"samples\": [\n                        0, \n                        1\n                      ], \n                      \"semantic_type\": \"\", \n                      \"description\": \"\"\n                    }\n                  }\n                }\n              }\n            }\n          }\n        }\n      }\n    }\n  }\n], \n\"type\": \"dataframe\", \"variable_name\": \"titanic_df\"}

```

```
titanic_df.tail()
```

```

{\"summary\": {\n  \"name\": \"titanic_df\", \n  \"rows\": 5, \n  \"fields\": [\n    {\n      \"column\": \"PassengerId\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 1, \n        \"min\": 887, \n        \"max\": 891, \n        \"num_unique_values\": 5, \n        \"samples\": [\n          888, \n          891, \n          889\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }, \n      {\n        \"column\": \"Survived\", \n        \"properties\": {\n          \"dtype\": \"category\", \n          \"num_unique_values\": 2, \n          \"samples\": [\n            0, \n            1\n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\"\n        }\n      }\n    }\n  ], \n  \"type\": \"dataframe\", \n  \"variable_name\": \"titanic_df\"}

```

```

\"Survived\", \n      \"properties\": { \n      \"dtype\":
\"number\", \n      \"std\": 0, \n      \"min\": 0, \n
\"max\": 1, \n      \"num_unique_values\": 2, \n      \"samples\":
[ \n      1, \n      0 \n      ], \n      \"semantic_type\":
\"\", \n      \"description\": \"\" \n      } \n      }, \n      { \n
\"column\": \"Pclass\", \n      \"properties\": { \n      \"dtype\":
\"number\", \n      \"std\": 1, \n      \"min\": 1, \n
\"max\": 3, \n      \"num_unique_values\": 3, \n      \"samples\":
[ \n      2, \n      1 \n      ], \n      \"semantic_type\":
\"\", \n      \"description\": \"\" \n      } \n      }, \n      { \n
\"column\": \"Name\", \n      \"properties\": { \n      \"dtype\":
\"string\", \n      \"num_unique_values\": 5, \n      \"samples\":
[ \n      \"Graham, Miss. Margaret Edith\", \n      \"Dooley,
Mr. Patrick\" \n      ], \n      \"semantic_type\": \"\", \n
\"description\": \"\" \n      } \n      }, \n      { \n      \"column\":
\"Sex\", \n      \"properties\": { \n      \"dtype\": \"category\", \n
\"num_unique_values\": 2, \n      \"samples\": [ \n
\"female\", \n      \"male\" \n      ], \n      \"semantic_type\": \"\", \n
\"description\": \"\" \n      } \n      }, \n      { \n      \"column\":
\"Age\", \n      \"properties\": { \n      \"dtype\": \"number\", \n
\"std\": 5.354126134736337, \n      \"min\": 19.0, \n      \"max\": 32.0, \n
\"num_unique_values\": 4, \n      \"samples\": [ \n
19.0, \n      32.0 \n      ], \n      \"semantic_type\": \"\", \n
\"description\": \"\" \n      } \n      }, \n      { \n      \"column\":
\"SibSp\", \n      \"properties\": { \n      \"dtype\": \"number\", \n
\"std\": 0, \n      \"min\": 0, \n      \"max\": 1, \n
\"num_unique_values\": 2, \n      \"samples\": [ \n
0 \n      ], \n      \"semantic_type\": \"\", \n
\"description\": \"\" \n      } \n      }, \n      { \n      \"column\":
\"Parch\", \n      \"properties\": { \n      \"dtype\": \"number\", \n
\"std\": 0, \n      \"min\": 0, \n      \"max\": 2, \n
\"num_unique_values\": 2, \n      \"samples\": [ \n
0 \n      ], \n      \"semantic_type\": \"\", \n
\"description\": \"\" \n      } \n      }, \n      { \n      \"column\":
\"Ticket\", \n      \"properties\": { \n      \"dtype\": \"string\", \n
\"num_unique_values\": 5, \n      \"samples\": [ \n
\"112053\", \n      \"370376\" \n      ], \n      \"semantic_type\": \"\", \n
\"description\": \"\" \n      } \n      }, \n      { \n      \"column\":
\"Fare\", \n      \"properties\": { \n      \"dtype\": \"number\", \n
\"std\": 10.09253436952285, \n      \"min\": 7.75, \n      \"max\": 30.0, \n
\"num_unique_values\": 4, \n      \"samples\": [ \n
30.0, \n      7.75 \n      ], \n      \"semantic_type\": \"\", \n
\"description\": \"\" \n      } \n      }, \n      { \n      \"column\":
\"Cabin\", \n      \"properties\": { \n      \"dtype\": \"category\", \n
\"num_unique_values\": 2, \n      \"samples\": [ \n
\"C148\", \n      \"B42\" \n      ], \n      \"semantic_type\":
\"\", \n      \"description\": \"\" \n      } \n      }, \n      { \n

```

```

{"column": "Embarked",\n      "properties": {\n        "dtype":\n        "string",\n        "num_unique_values": 3,\n        "samples":\n        [\n          "S",\n          "C"\n        ],\n        "semantic_type": "\",\n        "description": "\n      }\n    }\n  ],\n  "type": "dataframe"}

```

```
titanic_df.shape
```

```
(891, 12)
```

```
# Get a concise summary of the dataframe
```

```
print("\nDataset Information:")
```

```
titanic_df.info()
```

```
Dataset Information:
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 891 entries, 0 to 890
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
0	PassengerId	891 non-null	int64
1	Survived	891 non-null	int64
2	Pclass	891 non-null	int64
3	Name	891 non-null	object
4	Sex	891 non-null	object
5	Age	714 non-null	float64
6	SibSp	891 non-null	int64
7	Parch	891 non-null	int64
8	Ticket	891 non-null	object
9	Fare	891 non-null	float64
10	Cabin	204 non-null	object
11	Embarked	889 non-null	object

```
dtypes: float64(2), int64(5), object(5)
```

```
memory usage: 83.7+ KB
```

Interpretation of `.info()`:

- The dataset contains 891 entries (passengers) and 12 columns.
- **Missing Values Identified:** Age, Cabin, and Embarked have missing values. Cabin is missing a significant amount of data (~77%), which will require special attention.

```
# Get descriptive statistics for numerical columns
```

```
print("\nDescriptive Statistics:")
```

```
titanic_df.describe()
```

```
Descriptive Statistics:
```

```

{"summary": "{\n  "name": "titanic_df",\n  "rows": 8,\n  "fields": [\n    {\n      "column": "PassengerId",\n
```

```

{"properties": {"dtype": "number", "std": 320.8159711429856, "min": 1.0, "max": 891.0, "num_unique_values": 6, "samples": [891.0, 446.0, 668.5], "semantic_type": ""}, {"column": "Survived", "properties": {"dtype": "number", "std": 314.8713661874558, "min": 0.0, "max": 891.0, "num_unique_values": 5, "samples": [0.3838383838383838, 1.0, 0.4865924542648585], "semantic_type": ""}, {"column": "Pclass", "properties": {"dtype": "number", "std": 314.2523437079693, "min": 0.8360712409770513, "max": 891.0, "num_unique_values": 6, "samples": [891.0, 2.308641975308642, 3.0], "semantic_type": ""}, {"column": "Age", "properties": {"dtype": "number", "std": 242.9056731818781, "min": 0.42, "max": 714.0, "num_unique_values": 8, "samples": [29.69911764705882, 28.0, 714.0], "semantic_type": ""}, {"column": "SibSp", "properties": {"dtype": "number", "std": 314.4908277465442, "min": 0.0, "max": 891.0, "num_unique_values": 6, "samples": [891.0, 0.5230078563411896, 8.0], "semantic_type": ""}, {"column": "Parch", "properties": {"dtype": "number", "std": 314.65971717879, "min": 0.0, "max": 891.0, "num_unique_values": 5, "samples": [0.38159371492704824, 6.0, 0.8060572211299559], "semantic_type": ""}, {"column": "Fare", "properties": {"dtype": "number", "std": 330.6256632228577, "min": 0.0, "max": 891.0, "num_unique_values": 8, "samples": [32.204207968574636, 14.4542, 891.0], "semantic_type": ""}, {"description": ""}], "type": "dataframe"}

```

Interpretation of `.describe()`:

- **Survived:** About 38.4% of passengers in this dataset survived.
- **Age:** The age ranges from ~5 months to 80 years, with an average age of about 30.
- **Fare:** The fare is highly skewed, with a mean of \$32 but a median of only \$14.45. The maximum fare is over \$512, indicating the presence of extreme outliers.

```
titanic_df['Cabin'].value_counts()

Cabin
G6      4
C23 C25 C27  4
B96 B98    4
F2       3
D        3
..
E17      1
A24      1
C50      1
B42      1
C148     1
Name: count, Length: 147, dtype: int64
```

Step 3: Data Cleaning

Before analysis, we must handle the missing values we identified.

Theoretical Concept: Missing Value Imputation

Imputation is the process of replacing missing data with substituted values. The strategy depends on the data type and its distribution:

- **Numerical Data:** For skewed distributions (like `Age` and `Fare`), using the **median** is more robust than the mean because it is not affected by outliers.
- **Categorical Data:** A common strategy is to fill with the **mode** (the most frequent value).
- **High Cardinality/Too Many Missing Values:** For columns like `Cabin`, where most data is missing, imputing might not be effective. We could either drop the column or engineer a new feature from it (e.g., `Has_Cabin`).

```
titanic_df.isna().sum()

PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age          177
SibSp         0
Parch         0
Ticket        0
Fare          0
Cabin        687
Embarked       2
dtype: int64

print("Missing values before cleaning:")
titanic_df.isna().sum()
```

Missing values before cleaning:

```
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin            687
Embarked         2
dtype: int64
```

```
median = titanic_df['Age'].median()
print(median)
```

28.0

```
# 1. Handle missing 'Age' values
# We use the median to fill missing ages because the age distribution
can be skewed.
```

```
median_age = titanic_df['Age'].median()
titanic_df['Age'] = titanic_df['Age'].fillna(median_age)
```

```
# Verify that there are no more missing values in the columns we
handled so far
```

```
print("Missing values after Age cleaning:")
print(titanic_df[['Age', 'Embarked', 'Cabin']].isna().sum())
```

Missing values after Age cleaning:

```
Age           0
Embarked      2
Cabin        687
dtype: int64
```

```
mode = titanic_df['Embarked'].mode()[0]
print(mode)
```

S

```
# 2. Handle missing 'Embarked' values
# Since there are only two missing values, we'll fill them with the
most common port of embarkation (the mode).
```

```
mode_embarked = titanic_df['Embarked'].mode()[0]
titanic_df['Embarked'] = titanic_df['Embarked'].fillna(mode_embarked)
```

```
# Verify that there are no more missing values in the columns we
handled so far
```



```

print("Missing values after Embarked cleaning:")
print(titanic_df[['Age', 'Embarked', 'Cabin']].isna().sum())

Missing values after Embarked cleaning:
Age      0
Embarked  0
Cabin    687
dtype: int64

# 3. Handle the 'Cabin' column
# With over 77% missing data, imputing is not a good idea. Instead,
# we'll create a new feature 'Has_Cabin'.
titanic_df['Has_Cabin'] = titanic_df['Cabin'].notna().astype(int) # 1
if has cabin, 0 if not
titanic_df.drop('Cabin', axis=1, inplace=True) # Drop the original
column

```

titanic_df['Cabin'].notna(): This checks each value in the 'Cabin' column to see if it is not a missing value (NaN). It returns a boolean Series (True for non-missing, False for missing).

```

titanic_df['Has_Cabin'].value_counts()

Has_Cabin
0      687
1      204
Name: count, dtype: int64

titanic_df.head(5)

{"summary":{"\n  \"name\": \"titanic_df\",\n  \"rows\": 891,\n  \"fields\": [\n    {\n      \"column\": \"PassengerId\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 257,\n        \"min\": 1,\n        \"max\": 891,\n        \"num_unique_values\": 891,\n        \"samples\": [\n          710,\n          440,\n          841\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Survived\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n        \"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          1,\n          0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Pclass\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 1,\n        \"max\": 3,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          3,\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 891,\n        \"samples\": [\n          \"Moubarek, Master. Halim Gonios (\\\"William George\\\")\",\n          \"Kvillner, Mr. Johan Henrik Johannesson\"\n        ]

```



```
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
Has_Cabin      0
dtype: int64
```

Step 4: Univariate Analysis

We analyze each variable individually to understand its distribution.

Theoretical Concept: Univariate Analysis

This is the simplest form of data analysis, where the data being analyzed contains only one variable. The main purpose is to describe the data and find patterns within it.

- **For Categorical Variables:** We use frequency tables, bar charts (`countplot`), or pie charts to see the count or proportion of each category.
- **For Numerical Variables:** We use histograms (`histplot`) or kernel density plots (`kdeplot`) to understand the distribution, and box plots (`boxplot`) to identify the central tendency, spread, and outliers.

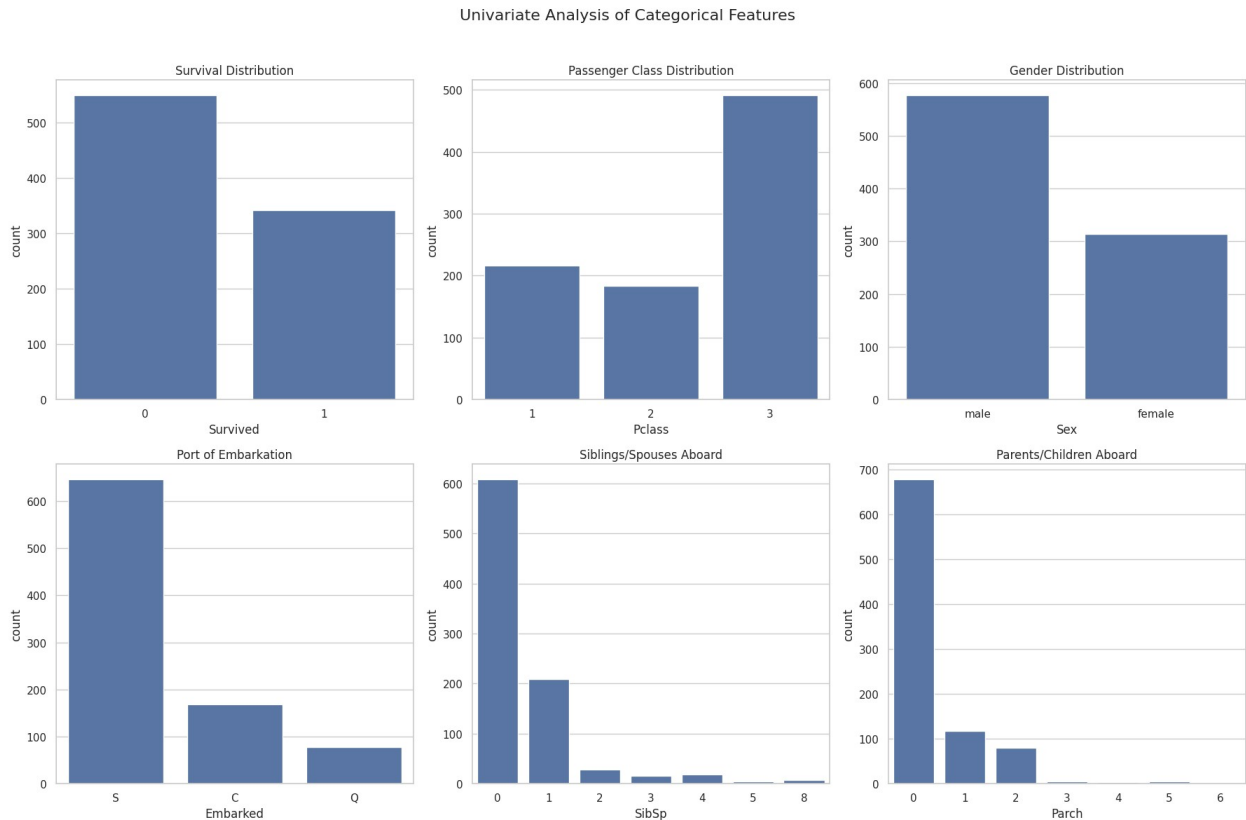
```
print("Analyzing categorical features:")

# Set up the figure for plotting
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Univariate Analysis of Categorical Features',
             fontsize=16)

# Plotting each categorical feature
sns.countplot(ax=axes[0, 0], x='Survived',
              data=titanic_df).set_title('Survival Distribution')
sns.countplot(ax=axes[0, 1], x='Pclass',
              data=titanic_df).set_title('Passenger Class Distribution')
sns.countplot(ax=axes[0, 2], x='Sex',
              data=titanic_df).set_title('Gender Distribution')
sns.countplot(ax=axes[1, 0], x='Embarked',
              data=titanic_df).set_title('Port of Embarkation')
sns.countplot(ax=axes[1, 1], x='SibSp',
              data=titanic_df).set_title('Siblings/Spouses Aboard')
sns.countplot(ax=axes[1, 2], x='Parch',
              data=titanic_df).set_title('Parents/Children Aboard')
```

```
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

Analyzing categorical features:



Key Insights (Categorical):

- **Survival:** Most passengers (over 500) did not survive.
- **Pclass:** The 3rd class was the most populated, followed by 1st and then 2nd.
- **Sex:** There were significantly more males than females.
- **Embarked:** The vast majority of passengers embarked from Southampton ('S').
- **SibSp & Parch:** Most passengers traveled alone.

```
print("\nAnalyzing numerical features:")

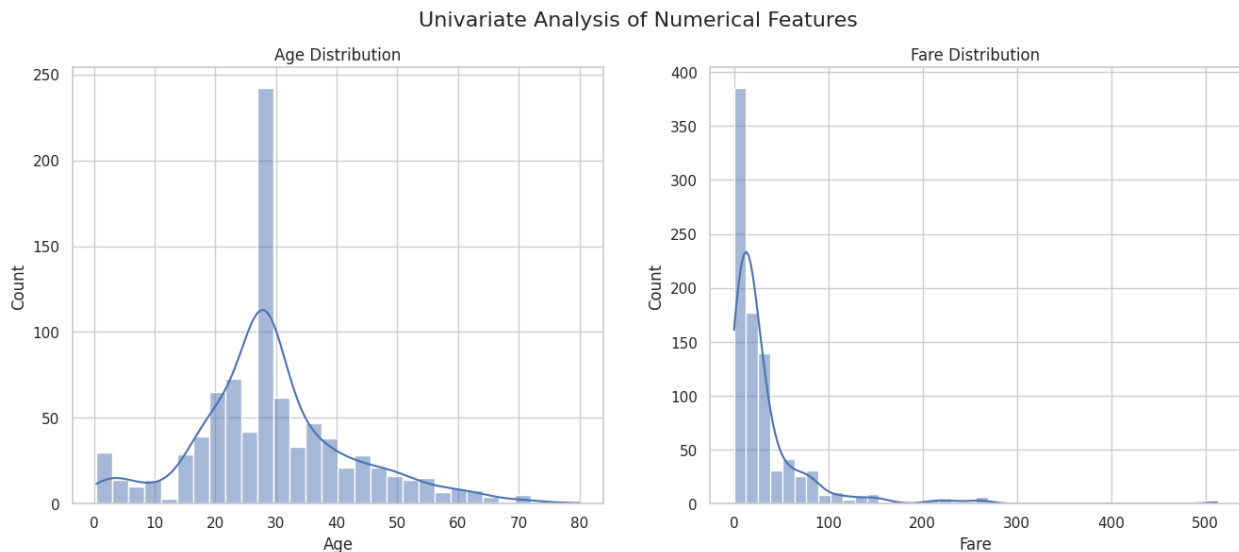
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Univariate Analysis of Numerical Features', fontsize=16)

# Plotting Age distribution
sns.histplot(ax=axes[0], data=titanic_df, x='Age', kde=True,
bins=30).set_title('Age Distribution')

# Plotting Fare distribution
sns.histplot(ax=axes[1], data=titanic_df, x='Fare', kde=True,
bins=40).set_title('Fare Distribution')
```

```
plt.show()
```

Analyzing numerical features:



Key Insights (Numerical):

- **Age:** The distribution peaks around the 20-30 age range. Remember we filled missing values with the median (28), which contributes to the height of that central bar.
- **Fare:** The distribution is heavily right-skewed, confirming that most tickets were cheap, with a few very expensive exceptions.

Step 5: Bivariate Analysis

Here, we explore the relationship between two variables. Our primary focus will be on how each feature relates to our target variable, Survived.

Theoretical Concept: Bivariate Analysis

This type of analysis involves two different variables, and its main purpose is to find relationships between them.

- **Categorical vs. Numerical:** To compare a numerical variable across different categories, we often use bar plots (`barplot`) that show the mean (or another estimator) of the numerical variable for each category. We can also use box plots or violin plots.
- **Categorical vs. Categorical:** We can use stacked bar charts or contingency tables (`crosstabs`).
- **Numerical vs. Numerical:** A scatter plot is the standard choice, with a correlation matrix being used to quantify the relationship.

```
print("Bivariate Analysis: Feature vs. Survival")
```

```
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
fig.suptitle('Bivariate Analysis with Survival', fontsize=16)

# Pclass vs. Survived
sns.barplot(ax=axes[0, 0], x='Pclass', y='Survived',
data=titanic_df).set_title('Survival Rate by Pclass')

# Sex vs. Survived
sns.barplot(ax=axes[0, 1], x='Sex', y='Survived',
data=titanic_df).set_title('Survival Rate by Sex')

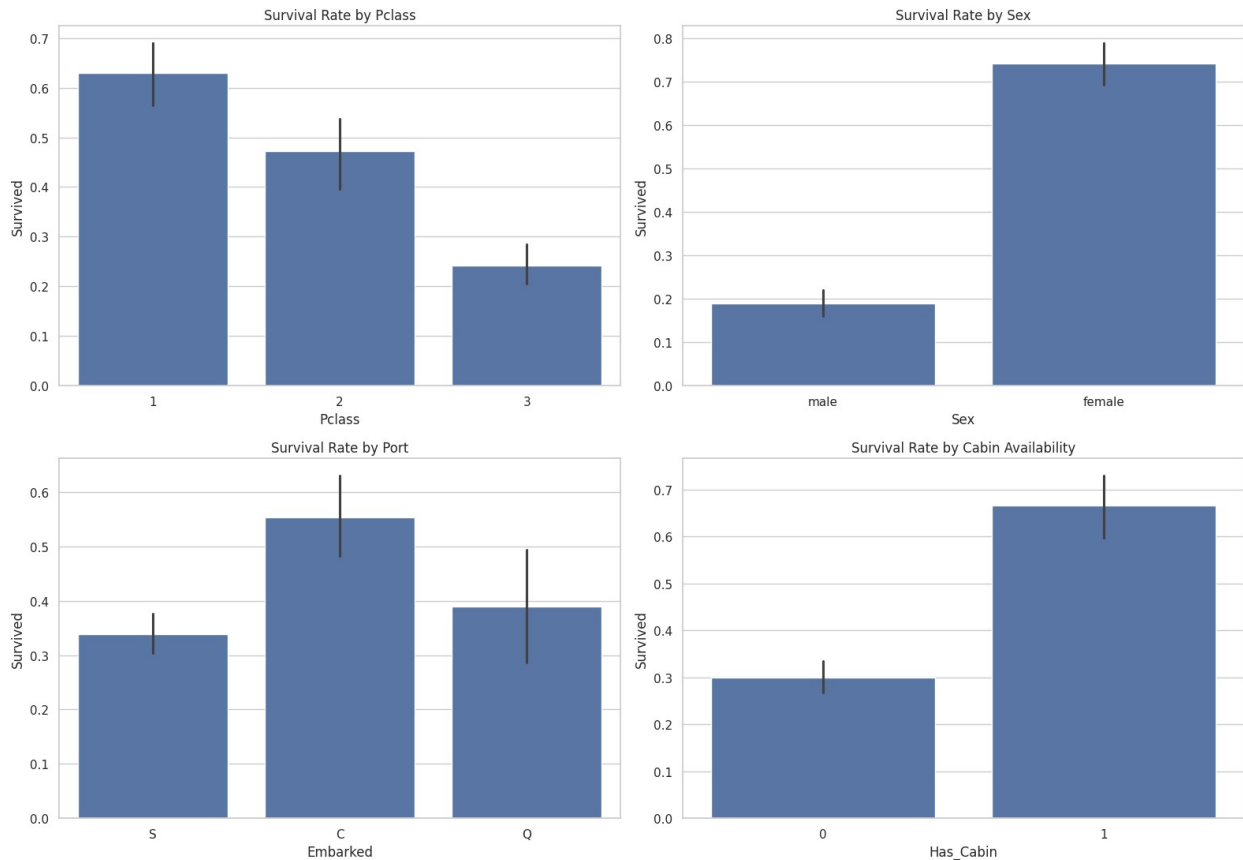
# Embarked vs. Survived
sns.barplot(ax=axes[1, 0], x='Embarked', y='Survived',
data=titanic_df).set_title('Survival Rate by Port')

# Has_Cabin vs. Survived
sns.barplot(ax=axes[1, 1], x='Has_Cabin', y='Survived',
data=titanic_df).set_title('Survival Rate by Cabin Availability')

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

Bivariate Analysis: Feature vs. Survival

Bivariate Analysis with Survival



Those black lines are called confidence intervals.

In a bar plot that shows the mean (like the survival rate here, which is the mean of the 0s and 1s in the 'Survived' column), the confidence interval represents the range within which the true mean is likely to fall. A shorter line indicates less uncertainty about the mean, while a longer line indicates more uncertainty.

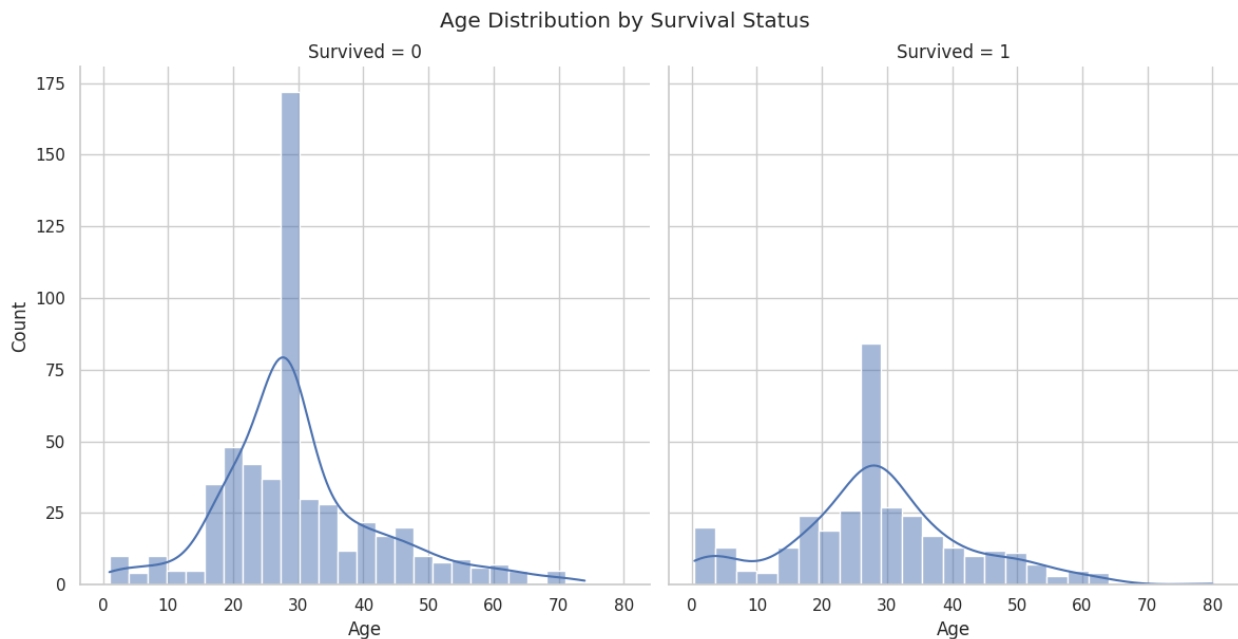
Seaborn automatically calculates and displays these 95% confidence intervals when you use `barplot`.

Key Insights (Bivariate):

- **Pclass:** A clear trend emerges: 1st class passengers had a >60% survival rate, while 3rd class passengers had less than 25%.
- **Sex:** This is the strongest predictor. Females had a survival rate of ~75%, while males had a rate below 20%.
- **Embarked:** Passengers embarking from Cherbourg ('C') had a higher survival rate than those from the other ports.
- **Has_Cabin:** Passengers with a registered cabin number had a much higher survival rate. This is likely correlated with being in 1st class.

```
# Age vs. Survival
g = sns.FacetGrid(titanic_df, col='Survived', height=6)
```

```
g.map(sns.histplot, 'Age', bins=25, kde=True)
plt.suptitle('Age Distribution by Survival Status', y=1.02)
plt.show()
```



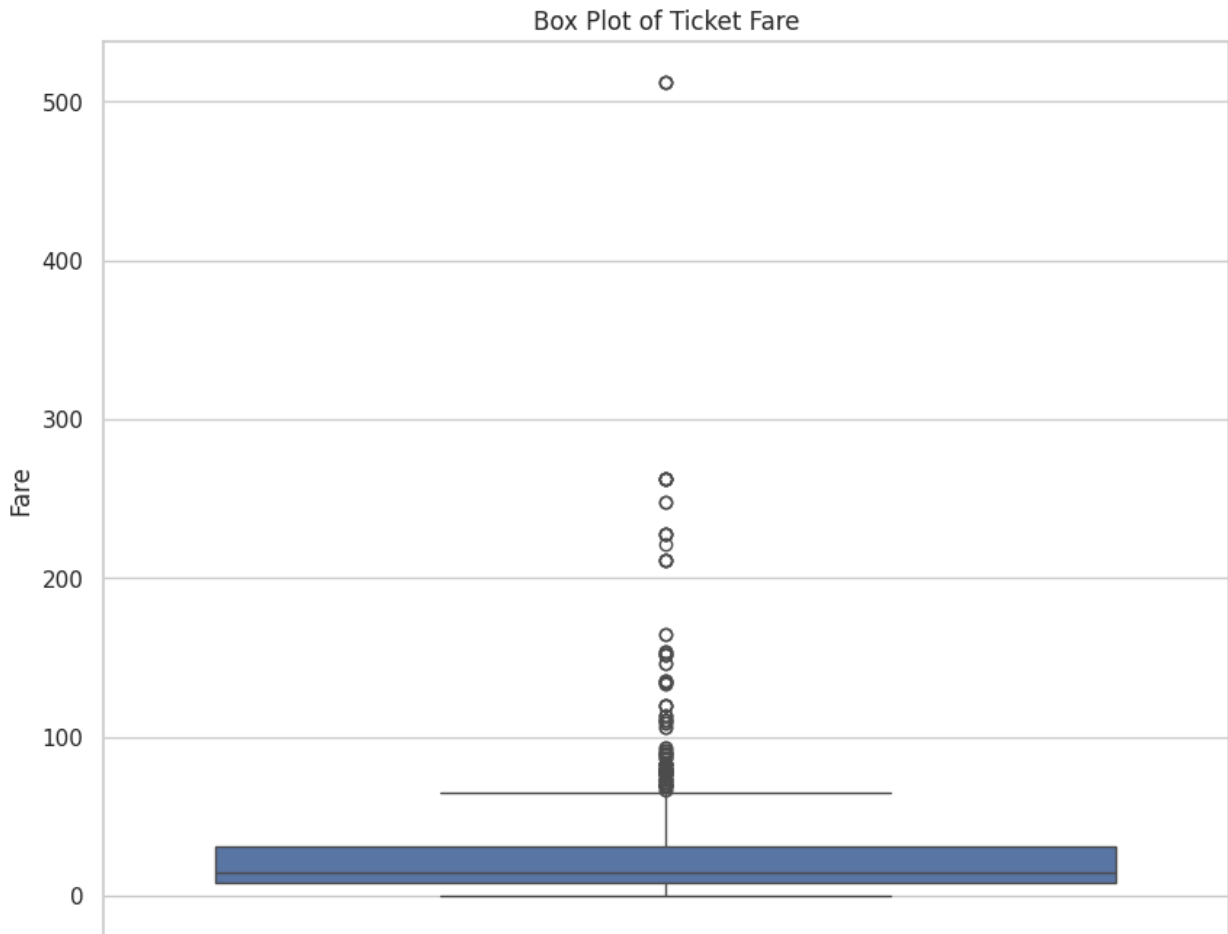
Key Insight (Age vs. Survival):

- Infants and young children had a higher probability of survival.
- A large portion of non-survivors were young adults (20-40).
- The oldest passengers (80 years) did not survive.

Deeper Dive: Outlier Analysis for 'Fare'

The `.describe()` function and histogram showed that `Fare` has extreme outliers. Let's visualize this clearly with a box plot.

```
plt.figure(figsize=(10, 8))
sns.boxplot(y='Fare', data=titanic_df)
plt.title('Box Plot of Ticket Fare')
plt.ylabel('Fare')
plt.show()
```

Observation: The box plot confirms the presence of significant outliers. Most fares are concentrated below \$100, but there are several fares extending far beyond, with some even exceeding \$500. These are likely first-class passengers who booked luxurious suites. For some machine learning models, handling these outliers (e.g., through log transformation) would be an important step.

Step 6: Feature Engineering

Now, we'll create new features from the existing ones to potentially uncover deeper insights and provide more useful information for a machine learning model.

Theoretical Concept: Feature Engineering

Feature engineering is the process of using domain knowledge to extract features (characteristics, properties, attributes) from raw data. A good feature should be relevant to the problem and easy for a model to understand.

Common Techniques:

1. **Combining Features:** Creating a new feature by combining others (e.g., `SibSp + Parch = FamilySize`).

2. **Extracting from Text:** Pulling out specific information from a text feature (e.g., extracting titles from the `Name` column).
3. **Binning:** Converting a continuous numerical feature into a categorical one (e.g., binning `Age` into groups like 'Child', 'Adult', 'Senior').

```
# 1. Create a 'FamilySize' feature
titanic_df['FamilySize'] = titanic_df['SibSp'] + titanic_df['Parch'] +
1 # +1 for the person themselves

# 2. Create an 'IsAlone' feature
titanic_df['IsAlone'] = 0
titanic_df.loc[titanic_df['FamilySize'] == 1, 'IsAlone'] = 1

print("Created 'FamilySize' and 'IsAlone' features:")
titanic_df[['FamilySize', 'IsAlone']].head()

Created 'FamilySize' and 'IsAlone' features:

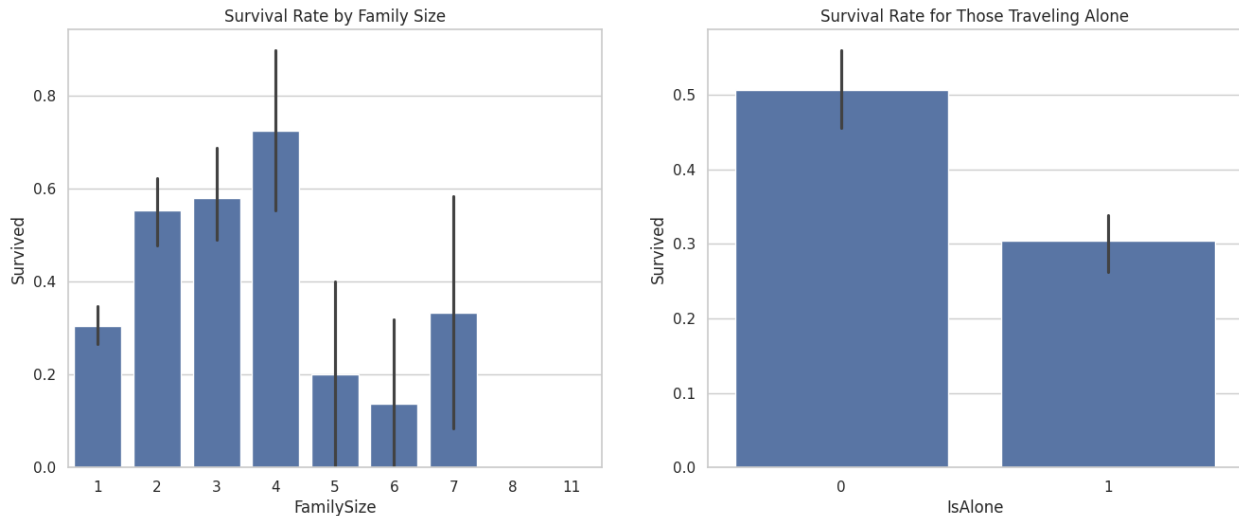
{"summary":{"\n  \"name\": \"titanic_df[['FamilySize', 'IsAlone']]\", \n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"FamilySize\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 1, \n        \"max\": 2, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          1, \n          2\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }, \n      {\n        \"column\": \"IsAlone\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 0, \n          \"min\": 0, \n          \"max\": 1, \n          \"num_unique_values\": 2, \n          \"samples\": [\n            1, \n            0\n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\"\n        }\n      }\n    ]\n  }, \"type\": \"dataframe\"}

# Analyze the new family-related features against survival
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Survival Rate by FamilySize
sns.barplot(ax=axes[0], x='FamilySize', y='Survived',
data=titanic_df).set_title('Survival Rate by Family Size')

# Survival Rate by IsAlone
sns.barplot(ax=axes[1], x='IsAlone', y='Survived',
data=titanic_df).set_title('Survival Rate for Those Traveling Alone')

plt.show()
```



Insight:

- Passengers who were alone (`IsAlone=1`) had a lower survival rate (~30%) than those in small families.
- Small families of 2 to 4 members had the highest survival rates.
- Very large families (5 or more) had a very poor survival rate. This might be because it was harder for large families to stay together and evacuate.
- Matches a space.
- Titles in the names are usually preceded by a space. `([A-Za-z]+)`: This is the capturing group.
- `[A-Za-z]+`: Matches one or more uppercase or lowercase letters. This captures the title itself (like Mr, Mrs, Miss, etc.).
- `\\.`: Matches a literal dot (.) which usually follows the title.

```
# 3. Extract 'Title' from the 'Name' column
titanic_df['Title'] = titanic_df['Name'].str.extract(r' ([A-Za-z]+)\.', expand=False)
```

```
# Let's see the different titles
print("Extracted Titles:")
titanic_df['Title'].value_counts()
```

Extracted Titles:

Title	Count
Mr	517
Miss	182
Mrs	125
Master	40
Dr	7
Rev	6
Col	2
Mlle	2
Major	2

```

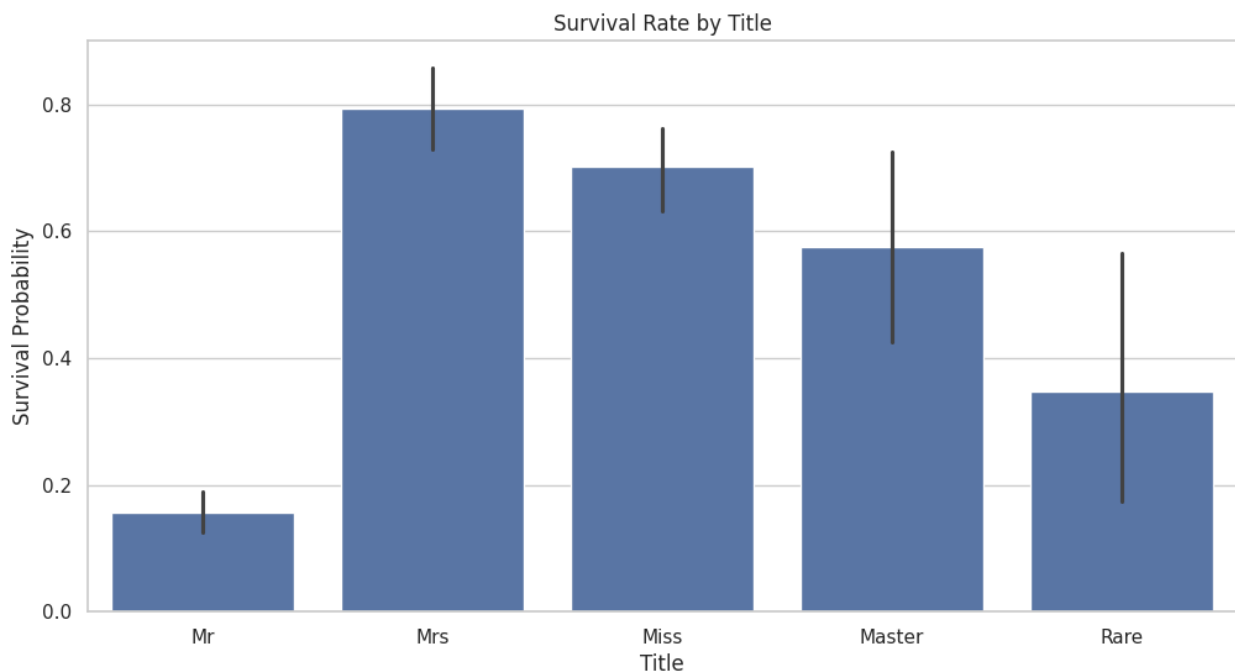
Ms          1
Mme         1
Don         1
Lady        1
Sir         1
Capt       1
Countess    1
Jonkheer    1
Name: count, dtype: int64

# Simplify the titles by grouping rare ones into a 'Rare' category
titanic_df['Title'] = titanic_df['Title'].replace(['Lady',
'Countess', 'Capt', 'Col', 'Don', 'Dr', 'Major', 'Rev', 'Sir',
'Jonkheer', 'Dona'], 'Rare')

titanic_df['Title'] = titanic_df['Title'].replace('Mlle', 'Miss')
titanic_df['Title'] = titanic_df['Title'].replace('Ms', 'Miss')
titanic_df['Title'] = titanic_df['Title'].replace('Mme', 'Mrs')

# Let's see the survival rate by the new, cleaned titles
plt.figure(figsize=(12, 6))
sns.barplot(x='Title', y='Survived', data=titanic_df)
plt.title('Survival Rate by Title')
plt.ylabel('Survival Probability')
plt.show()

```



Insight: The `Title` feature gives us powerful information. 'Mrs' and 'Miss' (females) had high survival rates. 'Mr' (males) had a very low survival rate. 'Master' (young boys) had a significantly higher survival rate than 'Mr', reinforcing the 'children first' idea. The 'Rare' titles, often

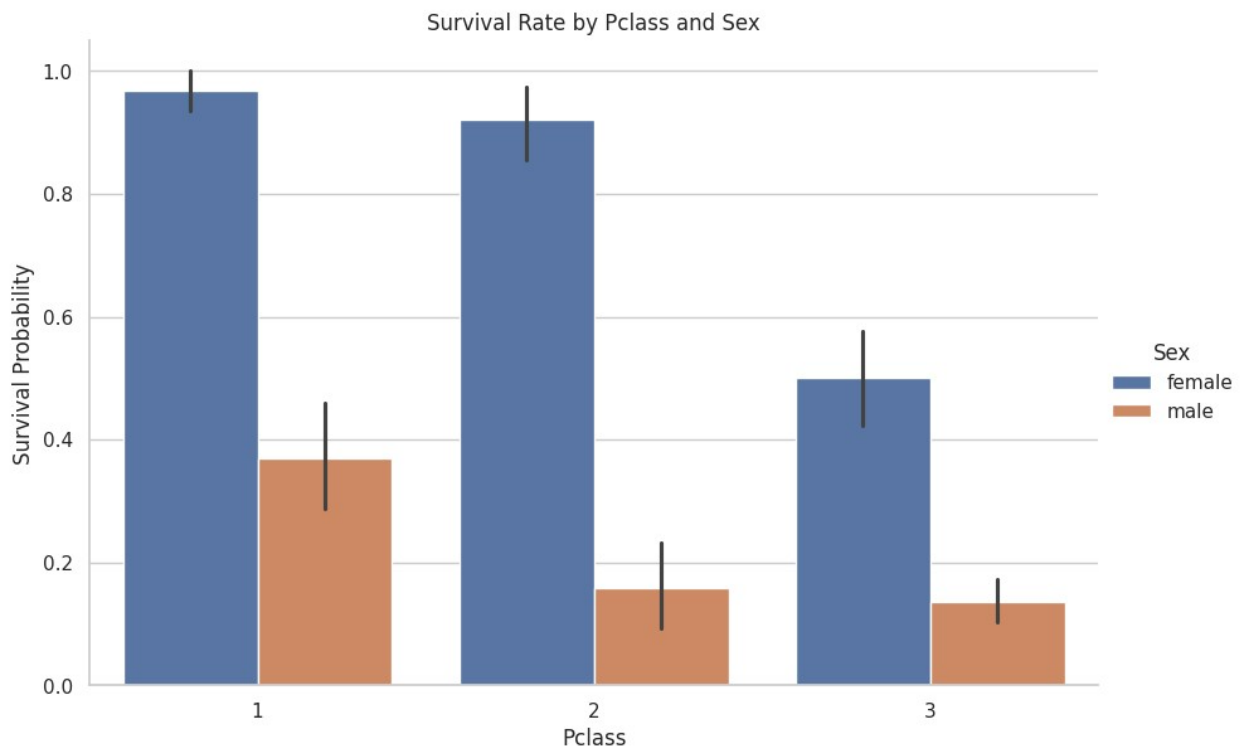
associated with nobility or status, also had a mixed but generally higher survival rate than common men.

Step 7: Multivariate Analysis

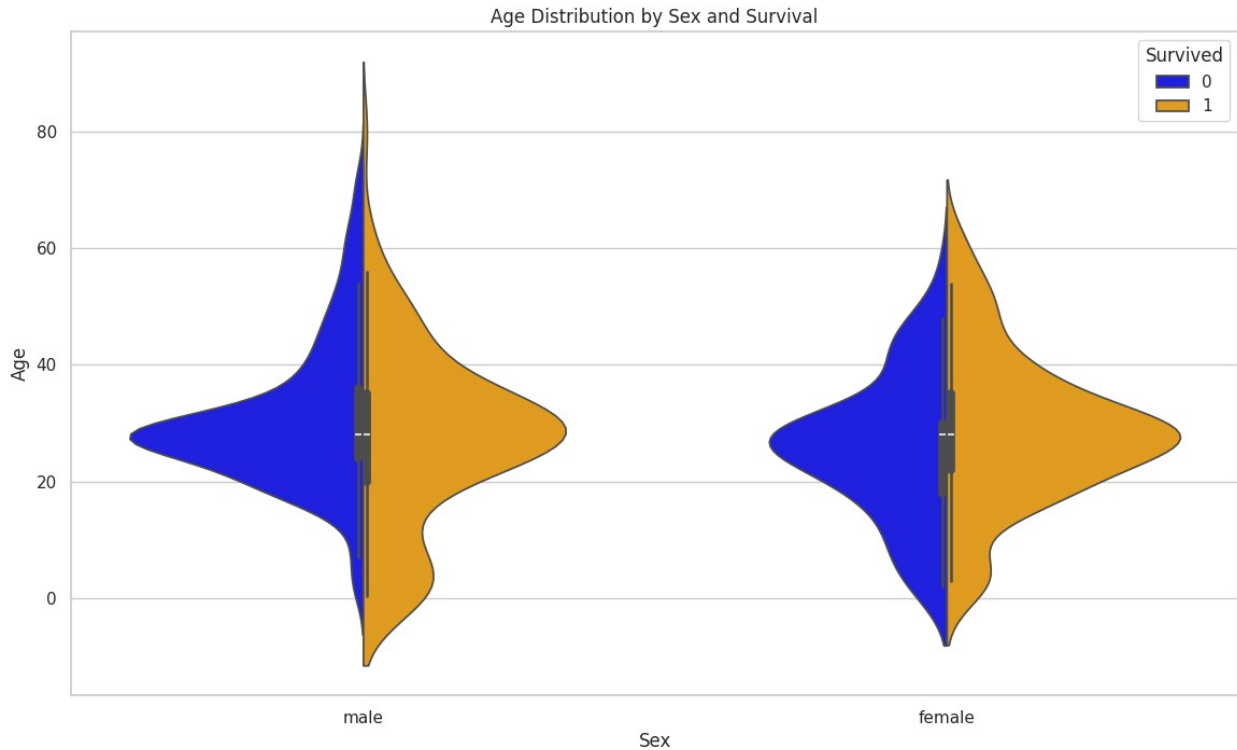
Now we explore interactions between multiple variables simultaneously, including our new engineered features.

```
# Survival rate by Pclass and Sex
sns.catplot(x='Pclass', y='Survived', hue='Sex', data=titanic_df,
kind='bar', height=6, aspect=1.5)
plt.title('Survival Rate by Pclass and Sex')
plt.ylabel('Survival Probability')
plt.show()

# Insights: Females in all classes had a significantly higher survival
rate than males.
```



```
# Violin plot to see age distribution by sex and survival status
plt.figure(figsize=(14, 8))
sns.violinplot(x='Sex', y='Age', hue='Survived', data=titanic_df,
split=True, palette={0: 'blue', 1: 'orange'})
plt.title('Age Distribution by Sex and Survival')
plt.show()
```

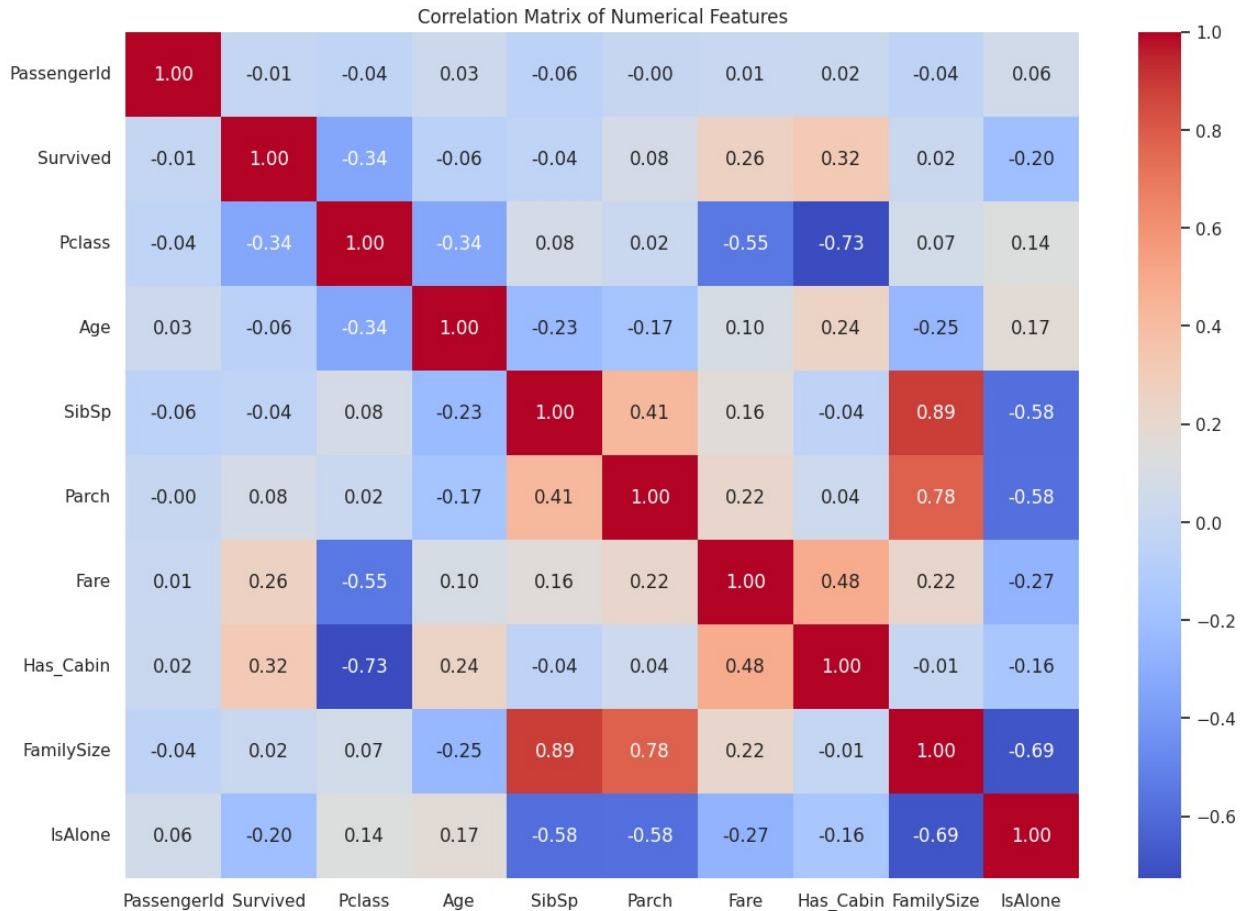


Insight from Violin Plot:

- For males, the peak of the distribution for survivors (orange) is at a very young age (children), while the peak for non-survivors is in the 20-30 range.
- For females, the distribution of survivors is much broader, indicating that females of most ages had a good chance of surviving.

Step 8: Correlation Analysis

```
# Correlation Heatmap for numerical features
plt.figure(figsize=(14, 10))
numeric_cols = titanic_df.select_dtypes(include=np.number)
correlation_matrix = numeric_cols.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm',
fmt='.2f')
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```



Interpretation of the Heatmap:

- **Survived** has a notable positive correlation with **Fare** and **Has_Cabin**, and a negative correlation with **Pclass** and our new **IsAlone** feature.
- **Pclass** and **Fare** are strongly negatively correlated, which makes sense (1st class = high fare).
- Our new **FamilySize** feature is composed of **SibSp** and **Parch**, so it's highly correlated with them by definition.

```
import pandas as pd

# Create a sample DataFrame
sample_data = {'col1': [1, 2, 3, 4],
               'col2': ['A', 'B', 'C', 'D'],
               'col3': [True, False, True, False]}
sample_df = pd.DataFrame(sample_data)

# Display the sample DataFrame
print("Sample DataFrame:")
display(sample_df)

Sample DataFrame:
```

```
{
  "summary": {
    "name": "sample_df",
    "rows": 4,
    "fields": [
      {
        "column": "col1",
        "properties": {
          "dtype": "number",
          "std": 1,
          "min": 1,
          "max": 4,
          "num_unique_values": 4,
          "samples": [2, 4, 1],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "col2",
        "properties": {
          "dtype": "string",
          "num_unique_values": 4,
          "samples": ["B", "D", "A"],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "col3",
        "properties": {
          "dtype": "boolean",
          "num_unique_values": 2,
          "samples": [false, true],
          "semantic_type": ""
        },
        "description": ""
      }
    ],
    "type": "dataframe",
    "variable_name": "sample_df"
  }
}
```

A sample dataframe is added here, do your analysis of titanic dataframe and provide us the y-profiling of that

```
# Install ydata-profiling
!pip install ydata-profiling -q

0:00:00 0.0/62.0 kB ? eta -:--:--
62.0/62.0 kB 4.6 MB/s eta
0:00:00 etadata (setup.py) ...
400.1/400.1 kB 25.7 MB/s eta 0:00:00
296.5/296.5 kB 22.6 MB/s eta
0:00:00
679.7/679.7 kB 40.1 MB/s eta
0:00:00
37.3/37.3 MB 46.3 MB/s eta
0:00:00
105.4/105.4 kB 8.4 MB/s eta
0:00:00
43.3/43.3 kB 3.3 MB/s eta
0:00:00
lmin (setup.py) ...

# Generate the profiling report
from ydata_profiling import ProfileReport

profile = ProfileReport(sample_df, title="Titanic Dataset Profiling Report")
```



```

# Display the report in the notebook
profile.to_notebook_iframe()

{"model_id":"fc702dbd13114245870c332df21f24f9","version_major":2,"version_minor":0}

100%|██████████| 3/3 [00:00<00:00, 59.82it/s]

{"model_id":"e385722b857944a1b93f00e4408483d7","version_major":2,"version_minor":0}

{"model_id":"536b1ca3f7a14d7fa5994091daf427a1","version_major":2,"version_minor":0}

<IPython.core.display.HTML object>

# Save the report to an HTML file
profile.to_file("sample.html")

{"model_id":"4c91fbd4aefc4073aa2b015afe4f081c","version_major":2,"version_minor":0}

```

Submission Instructions

For this assignment, you are required to submit the ydata-profiling data after completing the data cleaning steps.

1. **Run the ydata-profiling code:** Make sure you have run the code cells above to generate and display the profiling report in the notebook.
2. **Save your notebook:** Using the dummy code mentioned above, create the html report and share it with us".

Step 9: Final Conclusion and Summary of Insights

This end-to-end EDA has provided a deep understanding of the Titanic dataset. Our analysis confirms the "women and children first" narrative and highlights the stark social inequalities of the time. Through feature engineering, we've created even more powerful predictors for a potential machine learning model.

Key Findings:

1. **Strongest Predictors of Survival:**
 - **Title & Sex:** Being female ('Mrs', 'Miss') was the single most significant advantage. Our engineered `Title` feature captures this nuance better than `Sex` alone, also showing that young boys ('Master') had a much higher survival rate than adult men ('Mr').
 - **Passenger Class:** There was a clear survival hierarchy: 1st > 2nd > 3rd class.
 - **Age:** Children and infants had a higher survival rate.
2. **Other Influential Factors:**

- **Family Size:** Traveling in a small family (2-4 members) increased survival chances, while traveling alone or in a very large family decreased them.
- **Fare/Cabin:** Having a cabin (and thus paying a higher fare) was strongly correlated with survival, acting as a proxy for wealth and passenger class.
- **Port of Embarkation:** Passengers from Cherbourg ('C') had a higher survival rate, possibly because a higher proportion of them were in 1st class.

These insights are fundamental for the next step in the data science pipeline: building a predictive machine learning model to forecast survival.